OS UNIT 4 ORANGE PROBLEM

NAME: Sami Nisar Shahapuri SRN:PES1UG24CS611

SECTION: 4-K DATE: 16/04/2026

SCREENSHOTS:

1A

```
Sami@ubuntu2004:~/Desktop/orange_u4$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
Sami@ubuntu2004:~/Desktop/orange_u4$
```

1B

```
Sami@ubuntu2004:~/Desktop/orange_u4$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
Sami@ubuntu2004:~/Desktop/orange_u4$
```

2A

```
Sami@ubuntu2004:~/Desktop/orange_u4$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
Sami@ubuntu2004:~/Desktop/orange_u4$
```

2B

```
Sami@ubuntu2004:~/Desktop/orange_u4$ xxd .pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6 | head -20
00000000: 7472 6565 2039 3600 3130 3036 3434 2062  tree 96.100644 b
00000010: 7965 2e74 7874 00e6 7ed6 6bcb 4a70 8250  ye.txt..~.k.Jp.P
00000020: a89f 315d 4d8b b927 0334 3c84 df83 4438  ..1]M..'.4<...D8
00000030: 880e 13a6 8652 bc31 3030 3634 3420 6865  .....R.100644 he
00000040: 6c6c 6f2e 7478 7400 05c8 d777 d0be 8574  llo.txt....w...t
00000050: 8b67 88d1 b5ca f7d6 9754 e6e6 8060 c265  .g.......T...`.e
00000060: ca21 cbd6 c05e 5f43                      .!...^_C
Sami@ubuntu2004:~/Desktop/orange_u4$
```

3A

```
Sami@ubuntu2004:~/Desktop/orange_u4$ ./pes status
  untracked:  index.h
  untracked:  pes.h
  untracked:  object.c
  untracked:  tree.c
  untracked:  tree.h
  untracked:  test_tree.c
  untracked:  2nd.png
  untracked:  object.c.save.1
  untracked:  test_objects.c

Sami@ubuntu2004:~/Desktop/orange_u4$ ./pes add file1.txt file2.txt
Sami@ubuntu2004:~/Desktop/orange_u4$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt
  staged:     test.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  allphase1testspassed.png
  untracked:  object.c.save
  untracked:  object.c.backup
  untracked:  test_sequence.sh
  untracked:  README.md
  untracked:  commit.h
  untracked:  index.c
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  pes.c
  untracked:  Makefile
  untracked:  index.h
  untracked:  pes.h
  untracked:  object.c
  untracked:  tree.c
  untracked:  tree.h
  untracked:  test_tree.c
  untracked:  2nd.png
  untracked:  object.c.save.1
  untracked:  test_objects.c
```

3B

```
Sami@ubuntu2004:~/Desktop/orange_u4$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776344573 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776344582 6 file2.txt
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776344394 6 test.txt
Sami@ubuntu2004:~/Desktop/orange_u4$
```

4A

```
Sami@ubuntu2004:~/Desktop/orange_u4$ ./pes log
commit e5e5f6b9a93bc88530680ab4d69c3ca33f39af5e50ab65de903851182173184b
Author: PES User <pes@localhost>
Date:   1776346908

    Add farewelldd fA

commit 8ee0ac8f64bca8ace1c40d59e094f58fd62904ccb4fac69f45f3adaee8c5fbaa
Author: PES User <pes@localhost>
Date:   1776346866

    Add worldelldd fA

commit a08e56305334ac62e2ecca5d8471400a5ad48d32c90edae9087dd95f40e90e81
Author: PES User <pes@localhost>
Date:   1776346822

    Initial commit

Sami@ubuntu2004:~/Desktop/orange_u4$
```

4B

```
Sami@ubuntu2004:~/Desktop/orange_u4$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/8e/e0ac8f64bca8ace1c40d59e094f58fd62904ccb4fac69f45f3adaee8c5fbaa
.pes/objects/a0/8e56305334ac62e2ecca5d8471400a5ad48d32c90edae9087dd95f40e90e81
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e5/e5f6b9a93bc88530680ab4d69c3ca33f39af5e50ab65de903851182173184b
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
Sami@ubuntu2004:~/Desktop/orange_u4$
```

4C

```
Sami@ubuntu2004:~/Desktop/orange_u4$ cat .pes/refs/heads/main
e5e5f6b9a93bc88530680ab4d69c3ca33f39af5e50ab65de903851182173184b
Sami@ubuntu2004:~/Desktop/orange_u4$ cat .pes/HEAD
ref: refs/heads/main
Sami@ubuntu2004:~/Desktop/orange_u4$
```

FINAL REPORT

```
Sami@ubuntu2004:~/Desktop/orange u4$ make test-integration

--- Third Commit ---
Committed: 68040f1f1865... Add farewell

--- Full History ---
commit 68040f1f186537fd765ecea6cccd1dc75a7d00ae8654fa677524a44709dc31f1
Author: PES User <pes@localhost>
Date:   1776349367

    Add farewelldd fA

commit a294d976b2d13e786c316b21d2b060ffbb877c801aa780de964924e88e4103f3
Author: PES User <pes@localhost>
Date:   1776349367

    Update file.txtfA

commit c0dcff5372da494a77592ebcb3c367056d5edd7caebc34b537ae84bfd01394d8
Author: PES User <pes@localhost>
Date:   1776349367

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
68040f1f186537fd765ecea6cccd1dc75a7d00ae8654fa677524a44709dc31f1

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/68/040f1f186537fd765ecea6cccd1dc75a7d00ae8654fa677524a44709dc31f1
.pes/objects/a2/94d976b2d13e786c316b21d2b060ffbb877c801aa780de964924e88e4103f3
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c0/dcff5372da494a77592ebcb3c367056d5edd7caebc34b537ae84bfd01394d8
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
Sami@ubuntu2004:~/Desktop/orange u4$
```

Phase 5 & 6: Analysis-Only Questions

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

ANS

A branch is just a file under .pes/refs/heads/ containing a commit hash. To implement pes checkout <branch>, you would read that commit hash, read the commit object to get the root tree hash, then recursively walk the tree and write each blob to the working directory (creating directories, overwriting files). You must also update .pes/HEAD to point to ref: refs/heads/<branch> and update the index to match the new tree. The complexity comes from handling uncommitted changes (abort if a tracked file is dirty and differs between branches), performance (avoiding full tree walk every time), and deleting files that exist in the old branch but not in the new one.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

ANS

Using the index (which stores the hash, mtime, and size of each staged file), you can detect a dirty working directory by comparing each tracked file's current mtime and size with its index entry. If they differ, the file has unstaged changes. To decide whether checkout should refuse, you would then compare the blob hash of that file in the current commit's tree with the blob hash in the target branch's tree; if they differ, the dirty file would be overwritten, so checkout must abort. Additionally, if the target branch introduces a new file that already exists as an untracked file in the working directory, that also causes a conflict.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

ANS

In detached HEAD, .pes/HEAD contains a commit hash directly instead of a branch reference. If you make commits in this state, the new commits are created and HEAD moves to the new commit, but no branch points to them – they are only reachable via HEAD and the reflog. A user can recover those commits by creating a new branch that points to the commit hash (e.g., pes branch recover <hash>), which can be found by inspecting the reflog (a log of all HEAD movements) or by manually listing recent commit objects in the object store.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

ANS

To find and delete unreachable objects, you would perform a mark-and-sweep algorithm: first, traverse all references (branches, HEAD, tags) – for each commit, mark it, its tree, its parent commits, and recursively all trees and blobs reachable from those trees. Use a hash set (or a boolean array) to store marked object IDs efficiently. Then sweep through all files in .pes/objects/ and delete any object whose hash is not in the mark set. For a repository with 100,000 commits and 50 branches, you would need to visit every commit (100,000), every tree object (one per directory per commit – could be similar magnitude), and every blob (each unique file version). In practice, many objects are shared, but the algorithm still visits all reachable objects, which could be hundreds of thousands or millions.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

ANS

A dangerous race condition occurs when GC starts by reading the current set of references (branches) and marking reachable objects, while a concurrent commit writes new objects and updates a branch reference. If GC finishes marking before the commit updates the branch, it will not mark the new commit’s objects, and then during the sweep phase it may delete those newly written objects, corrupting the repository. Git avoids this by using a “grace period” – GC does not delete loose objects younger than a certain age (e.g., 2 weeks), by using atomic reference updates with rename() (which makes the reference change atomic), and by relying on the fact that objects are written before the reference is updated, so there is no window where a referenced object is missing. Additionally, Git’s git gc is typically run when the repository is idle or uses reachability bitmaps to reduce the risk.